

Tarea #03

Carrera:	Ingeniería de Software con IA	Semestre:	IV
Curso:	Backend Developer Web		
Instructor:	Moisés García		

Caso:

La empresa **TecnoStore S.A.C.** vende productos tecnológicos como audífonos, teclados, mouse, cargadores, cámaras web y memorias USB. Actualmente, el área de ventas lleva el control de productos en hojas de cálculo, lo que genera errores frecuentes: productos duplicados, precios desactualizados, dificultades para buscar información y pérdida de tiempo al modificar datos.

La gerencia ha solicitado desarrollar un pequeño módulo web en **PHP** que permita gestionar productos desde el navegador. El sistema debe permitir buscar productos por descripción, agregar nuevos productos, editar información existente y eliminar registros cuando ya no correspondan.

El equipo de desarrollo propone organizar la solución usando archivos PHP separados para cada operación, una clase Producto encargada de manejar los datos y una conexión a base de datos mediante **PDO**. Sin embargo, antes de programar, se requiere que los estudiantes analicen la estructura del sistema, identifiquen los elementos técnicos necesarios y propongan una solución ordenada, segura y mantenible.

Actividad 1: Identificación de archivos y responsabilidades

Un sistema de productos en PHP puede estar dividido en los siguientes archivos:

Archivo	Posible responsabilidad
index.php	
agregar.php	
editar.php	
eliminar.php	
producto.php	
conexion.php	

Completa la tabla indicando qué función debería cumplir cada archivo dentro del sistema.

Luego responde:

- ¿Por qué es mejor dividir el sistema en varios archivos en lugar de colocar todo en uno solo?
- ¿Qué archivo debería encargarse de mostrar la lista de productos?
- ¿Qué archivo debería contener la clase Producto?
- ¿Qué archivo debería encargarse únicamente de la conexión con MySQL?

- ¿Qué riesgos existen si se mezcla la conexión, la lógica de negocio y el diseño HTML en un solo archivo?

Actividad 2: Organización por capas

Relaciona cada elemento con la capa que corresponde:

Elemento	Capa sugerida: Vista / Modelo / Configuración
Formulario para buscar productos	
Clase Producto	
Archivo de conexión a MySQL	
Tabla HTML de resultados	
Método setInsertar()	
Método getBuscarPorDescripcion()	

Preguntas de análisis

- ¿Qué ventaja tiene separar la vista del modelo?
- ¿Por qué la conexión a la base de datos no debería repetirse en cada archivo?
- ¿Cómo ayuda esta organización al mantenimiento del sistema?

Preguntas de pensamiento crítico

- Si el sistema funciona aunque todo esté en un solo archivo, ¿sería correcto dejarlo así? Justifica.
- ¿Qué problemas aparecerían si en el futuro se agregan clientes, ventas y categorías?
- ¿Qué decisión técnica permitiría que otro programador entienda el sistema con mayor facilidad?

Esquema solicitado

Elabora un **esquema por capas** del sistema de productos. Debe incluir:

- Usuario.
- Formularios PHP.
- Clase Producto.
- Conexión PDO.
- Base de datos MySQL.
- Resultado mostrado en pantalla.

Actividad 3: Reconocimiento de controles de formulario

Un formulario de producto tiene los siguientes campos:

Campo del formulario	Nombre del control	Dato que almacena
Código del producto	txtId	
Descripción	txtDescripcion	

Categoría	txtCategoria	
Precio	txtPrecio	
Texto de búsqueda	txtDescripcionBuscar	

Completa la tabla indicando qué dato representa cada control.

Luego responde:

- ¿Para qué sirve el atributo name en un formulario HTML?
- ¿Por qué el campo código podría estar marcado como readonly?
- ¿Qué diferencia hay entre un campo de texto para buscar y un campo de texto para registrar?
- ¿Qué validaciones deberían aplicarse al precio?

Actividad 4: Envío de datos mediante GET

Analiza la siguiente situación:

Un formulario envía los datos del producto a través de la URL usando el método GET. Luego PHP recibe esos datos mediante \$_GET.

Completa la tabla:

Dato recibido	Variable PHP sugerida	Validación necesaria
txtId		
txtDescripcion		
txtCategoria		
txtPrecio		
txtDescripcionBuscar		

Responde:

- ¿Qué es \$_GET en PHP?
- ¿Qué significa verificar isset(\$_GET["txtId"])?
- ¿Qué ocurre si se intenta leer un dato que no fue enviado?
- ¿Qué diferencia existe entre usar GET y POST?
- ¿Qué método sería más recomendable para registrar o actualizar productos? Justifica.

Preguntas de análisis

- ¿Por qué es importante verificar si una variable existe antes de usarla?
- ¿Qué errores podrían aparecer si el usuario deja campos vacíos?
- ¿Por qué no se debe confiar completamente en los datos enviados desde un formulario?

Preguntas de pensamiento crítico

- Si el precio se envía por la URL, ¿qué riesgo podría existir?

- ¿Sería correcto permitir que el usuario modifique el código del producto desde el formulario? Justifica.
- ¿Qué impacto tendría registrar productos sin validar descripción, categoría y precio?

Esquema solicitado

Elabora un **diagrama de flujo** del envío de datos:

Formulario HTML → Usuario escribe datos → Envía formulario → PHP recibe con \$_GET → Verifica con isset() → Procesa información.

Actividad 5: Análisis de la clase Producto

Una clase Producto puede tener las siguientes propiedades:

Propiedad	Descripción	Tipo de dato sugerido
id		
descripcion		
categoria		
precio		

Completa la tabla.

Luego responde:

- ¿Qué representa una clase en PHP?
- ¿Qué representa un objeto?
- ¿Por qué tendría sentido crear una clase llamada Producto?
- ¿Qué relación existe entre la tabla producto de la base de datos y la clase Producto en PHP?

Actividad 6: Métodos de la clase

La clase Producto contiene métodos relacionados con el CRUD:

Método	Operación que realiza	Tipo de operación CRUD
getBuscarPorId(\$idBuscar)		
getBuscarPorDescripcion(\$descripcionBuscar)		
setInsertar(\$producto)		
setActualizar(\$producto)		
setEliminar(\$idEliminar)		

Completa la tabla.

Responde:

- ¿Por qué algunos métodos empiezan con get?
- ¿Por qué algunos métodos empiezan con set?
- ¿Qué método se usaría para mostrar los datos antes de editarlos?
- ¿Qué método se usaría para guardar un producto nuevo?
- ¿Qué método se usaría para eliminar un producto?

Preguntas de análisis

- ¿Por qué es conveniente que la clase Producto concentre las operaciones relacionadas con productos?
- ¿Qué pasaría si la búsqueda, inserción, actualización y eliminación estuvieran dispersas en varios archivos?
- ¿Qué ventaja ofrece trabajar con objetos en lugar de solo variables sueltas?

Preguntas de pensamiento crítico

- ¿Es recomendable declarar las propiedades como públicas? ¿Qué alternativa sería más adecuada desde la programación orientada a objetos?
- ¿Por qué una clase debería representar tanto datos como comportamiento?
- ¿Qué mejoras aplicarías a la clase Producto para que sea más segura y ordenada?

Esquema solicitado

Elabora un **diagrama de clase simple** para Producto, incluyendo:

- Propiedades.
- Métodos.
- Relación con la tabla producto.
- Relación con la conexión PDO.

Actividad 7: Interpretación de la conexión

Un sistema PHP puede obtener la conexión a MySQL mediante un método como:

```
$cnx = Conexion::getConexionMySQL();
```

Responde:

- ¿Qué representa \$cnx?
- ¿Qué significa llamar a un método usando ::?
- ¿Por qué el método de conexión podría ser estático?
- ¿Qué ventaja tiene centralizar la conexión en una clase Conexion?
- ¿Qué datos básicos se necesitan para conectar PHP con MySQL?

Completa:

Dato necesario	Ejemplo
Servidor	
Base de datos	

Usuario	
Contraseña	
Juego de caracteres	

Actividad 8: PDO y manejo de errores

Responde:

- ¿Qué es PDO?
- ¿Qué ventaja tiene PDO frente a escribir consultas de forma directa e insegura?
- ¿Por qué es recomendable usar excepciones para manejar errores?
- ¿Qué tipo de errores podrían ocurrir durante la conexión?
- ¿Qué mensaje debería ver el usuario si la conexión falla?

Completa la tabla:

Error posible	Causa probable	Acción recomendada
No conecta con MySQL		
Base de datos inexistente		
Usuario incorrecto		
Contraseña incorrecta		
Error de codificación de caracteres		

Preguntas de análisis

- ¿Qué diferencia existe entre un error de conexión y un error de consulta?
- ¿Por qué no se debe mostrar al usuario final toda la información técnica del error?
- ¿Qué ventaja tiene reutilizar una misma conexión en diferentes operaciones?

Preguntas de pensamiento crítico

- Si una conexión falla, ¿el sistema debería detenerse o continuar? Justifica según el caso.
- ¿Qué información técnica podría ser peligrosa si se muestra en pantalla?
- ¿Qué impacto tendría un sistema de ventas que no controle adecuadamente sus errores de base de datos?

Esquema solicitado

Elabora un **mapa de proceso de conexión PDO**:

Archivo PHP → Clase Conexion → Método de conexión → Objeto PDO → Base de datos MySQL → Respuesta de conexión.

Actividad 9: Análisis de búsqueda por código

Observa la siguiente estructura lógica:

```
$snt = $cnx->prepare("SELECT * FROM producto WHERE id = :idBuscar");
$snt->bindValue(":idBuscar", $idBuscar);
$snt->execute();
$fila = $snt->fetch();
```

Responde:

- ¿Qué hace prepare()?
- ¿Qué representa :idBuscar?
- ¿Qué hace bindValue()?
- ¿Qué hace execute()?
- ¿Qué hace fetch()?
- ¿Qué diferencia hay entre preparar una consulta y ejecutarla?

Completa:

Elemento	Función
\$snt	
prepare()	
:idBuscar	
bindValue()	
execute()	
fetch()	

Actividad 10: Búsqueda por descripción

Una búsqueda por descripción puede usar una consulta similar:

```
SELECT * FROM producto
WHERE descripcion LIKE CONCAT('%', :descripcionBuscar, '%');
```

Responde:

- ¿Para qué sirve LIKE?
- ¿Para qué se usa % en una búsqueda?
- ¿Qué significa buscar coincidencias parciales?
- ¿Por qué conviene usar parámetros incluso en búsquedas?
- ¿Qué resultado se espera si el usuario escribe solo una parte del nombre del producto?

Completa la tabla:

Texto ingresado por el usuario	Resultado esperado
mouse	
tecl	
usb	
Texto vacío	
Texto inexistente	

Preguntas de análisis

- ¿Cuál es la diferencia entre `fetch()` y `fetchAll()`?
- ¿Por qué una búsqueda por código devuelve normalmente un solo registro?
- ¿Por qué una búsqueda por descripción puede devolver varios registros?
- ¿Qué ocurre si no se encuentra ningún producto?

Preguntas de pensamiento crítico

- ¿Qué riesgo existe si se arma la consulta SQL concatenando directamente lo que escribe el usuario?
- ¿Por qué las consultas preparadas ayudan a prevenir inyección SQL?
- ¿Es suficiente usar `bindValue()` o también se deben validar los datos? Justifica.

Esquema solicitado

 Elabora un **esquema comparativo**:

Consulta insegura	Consulta segura con PDO
Datos se concatenan directamente	Datos se envían como parámetros
Mayor riesgo de inyección SQL	Menor riesgo de inyección SQL
Difícil de mantener	Más ordenada y controlada

Agrega flechas o pasos que muestren cómo viajan los datos desde el formulario hasta la consulta.

Actividad 11: Relacionar acciones con sentencias SQL

Completa la tabla:

Acción del usuario	Operación CRUD	Sentencia SQL principal	Método sugerido en la clase
Buscar productos por descripción			
Buscar producto por ID			
Agregar producto			
Editar producto			
Eliminar producto			

Actividad 12: Análisis de cada operación

Responde para cada operación:

Operación	¿Qué datos necesita?	¿Qué validaciones requiere?	¿Qué podría fallar?

Buscar por descripción			
Buscar por ID			
Insertar producto			
Actualizar producto			
Eliminar producto			

Preguntas de análisis

- ¿Por qué insertar un producto no debería incluir manualmente el código si este es autogenerado?
- ¿Por qué después de insertar puede ser necesario recuperar el nuevo ID?
- ¿Qué diferencia existe entre actualizar todos los datos del producto y actualizar solo el precio?
- ¿Qué precauciones deben tomarse antes de eliminar un producto?

Preguntas de pensamiento crítico

- ¿Sería mejor eliminar físicamente un producto o marcarlo como inactivo? Justifica.
- ¿Qué pasaría si se elimina un producto que ya fue usado en una venta?
- ¿Qué operación CRUD consideras más crítica para controlar y por qué?
- ¿Qué validaciones mínimas debería tener el sistema antes de insertar o actualizar?

Esquema solicitado

Elabora un **diagrama CRUD del módulo producto**:

- Crear: agregar producto.
- Leer: buscar por ID y buscar por descripción.
- Actualizar: editar producto.
- Eliminar: eliminar producto.

Relaciona cada acción con su método y su sentencia SQL.

Actividad 13: Análisis de navegación

Un sistema básico de productos puede tener enlaces como:

Enlace o botón	Acción esperada
Agregar	
Editar	
Eliminar	
Guardar	
Retornar	
Buscar	

Completa la tabla.

Luego responde:

- ¿Qué diferencia hay entre un enlace y un botón de formulario?
- ¿Por qué el enlace Editar debe enviar el ID del producto?
- ¿Por qué el enlace Eliminar debe confirmar la acción antes de ejecutarse?
- ¿Qué debería ocurrir después de guardar un producto?
- ¿Qué debería ocurrir después de eliminar un producto?

Actividad 14: Confirmación con JavaScript

Analiza esta situación:

Antes de eliminar un producto, el sistema muestra un mensaje:

“¿Está seguro que desea eliminar?”

Responde:

- ¿Por qué es importante confirmar una eliminación?
- ¿Qué podría pasar si el usuario elimina por error?
- ¿Qué función cumple JavaScript en esta parte del sistema?
- ¿La confirmación en JavaScript reemplaza las validaciones del servidor? Justifica.
- ¿Qué validaciones debería hacer PHP aunque JavaScript confirme la acción?

Preguntas de análisis

- ¿Por qué una operación de eliminación requiere más cuidado que una búsqueda?
- ¿Qué información debería mostrarse al usuario antes de confirmar?
- ¿Por qué el sistema debe redireccionar luego de eliminar?

Preguntas de pensamiento crítico

- ¿Es suficiente preguntar “¿Está seguro?” antes de eliminar? ¿Qué otra información debería mostrarse?
- ¿Qué pasaría si un usuario manipula la URL y envía manualmente un ID para eliminar?
- ¿Qué controles de seguridad faltan en un sistema básico antes de permitir eliminar registros?

Esquema solicitado

Elabora un **diagrama de navegación** entre páginas:

index.php → agregar.php → guardar → retornar

index.php → editar.php?id=... → actualizar → retornar

index.php → confirmar eliminación → eliminar.php?id=... → redireccionar a index.php

4. Caso de pensamiento crítico

Lee la siguiente afirmación:

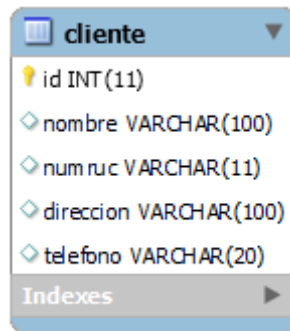
“Como el sistema es pequeño, no importa si los datos se envían por GET, si las propiedades son públicas y si se elimina directamente el producto de la base de datos. Lo importante es que el sistema funcione.”

Responde:

- ¿Qué errores técnicos presenta esta afirmación?
- ¿Por qué un sistema pequeño también debe desarrollarse con buenas prácticas?
- ¿Qué riesgo existe al enviar datos sensibles por GET?
- ¿Qué problema puede generar el uso de propiedades públicas sin validación?
- ¿Por qué eliminar directamente un producto puede afectar la integridad de la información?
- ¿Qué mejoras propondrías para hacer el sistema más seguro y mantenible?
- ¿Qué diferencia existe entre un sistema que “funciona” y un sistema correctamente diseñado?

Actividad 15: Desarrollo en PHP – PDO con POO

Realizar el desarrollo del CRUD para la Tabla **Cliente** utilizando **Programación Orientada a Objetos**



Entregables:

- Informe en Word con las Actividades desarrolladas.
- Código fuente en ZIP
- En el informe, código fuente y pantallazos de la ejecución del programa.

Todos los archivos deberán ser subidos al Blackboard.